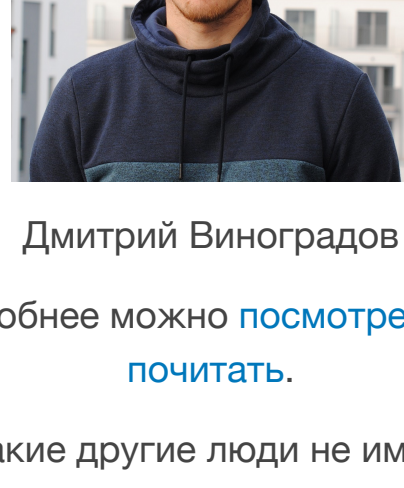


## Language



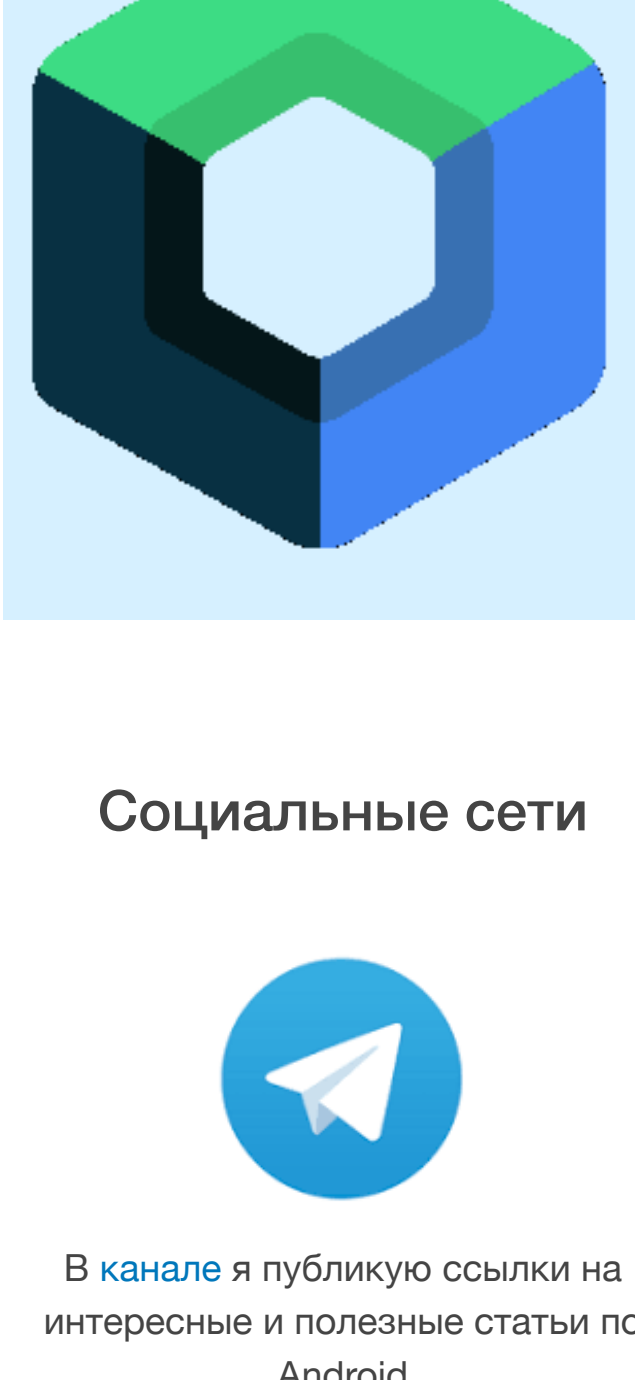
### Автор сайта



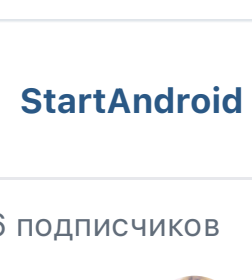
Дмитрий Виноградов

Подробнее можно [посмотреть](#) или [почитать](#).

Никакие другие люди не имеют к этому сайту никакого отношения и просто занимаются плагиатом.



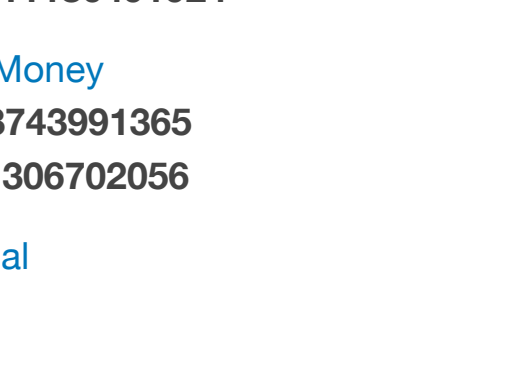
### Социальные сети



В канале я публикую ссылки на интересные и полезные статьи по Android

В чате можно обсудить вопросы и проблемы, возникающие при разработке

### Группа ВКонтакте



## Поддержка проекта

Яндекс  
410011180491924

WebMoney  
R246743991365  
Z551306702056

Paypal

## Урок 6. State

14 декабря 2022

В этом уроке мы разбираемся как такое State. Как он помогает Composable функции понять, что пора обновлять данные на экране.

В прошлых уроках мы создавали простые Composable функции.

Вот пример:

```
import androidx.compose.runtime.Composable
import androidx.compose.material.Text

@Composable
fun HomeScreen() {
    Text(text = "Hello Compose")
}
```

Она не принимает на вход никаких параметров и всегда выводит один и тот же текст на экран.

И этого, конечно, нам недостаточно. Мы хотели бы передавать на вход данные, чтобы текст отображал не статiku, а то, что нам нужно в момент вызова функции.

Мы можем добавить входной параметр в нашу функцию:

```
@Composable
fun HomeScreen(name: String) {
    Text(text = "Hello $name")
}
```

Этот параметр используем в Text, и получаем нужный нам текст на экране

В Activity вызов будет выглядеть так:

```
class MainActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContent {
            HomeScreen("Android")
        }
    }
}
```

В результате мы увидим текст Hello Android.

Тут все просто и понятно. Но в реальном приложении обычно все сложнее. Не всегда у нас есть нужные данные в момент вызова Composable функции. Или данные могут поменяться уже после вызова. Чтобы нам увидеть новые данные на экране, необходимо снова вызвать функцию и передать ей эти данные. Для решения этой задачи в Compose есть специальный механизм State. Рассмотрим его работу на примере счетчика кликов: нажимаем на текст и он показывает сколько всего раз мы нажали.

Если у вас был Preview для HomeScreen, то пока прокомментируйте или удалите его.

### Счетчик кликов

Начнем создание счетчика с простого нерабочего варианта:

```
@Composable
fun HomeScreen(count: Int) {
    Text(text = "Clicks: $count")
}
```

Получаем число кликов и выводим их на экран

Код Activity:

```
class MainActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        var count = 0
        setContent {
            HomeScreen(count)
        }
    }
}
```

Создам переменную-счетчик и передам ее значение в функцию.

Такой счетчик всегда показывает ноль сколько бы мы не нажимали. Как минимум потому, что мы нигде не ловим и не обрабатываем нажатия.

Давайте добавим обработчик кликов. Для элемента Text это можно сделать с помощью Modifier.clickable:

```
import androidx.compose.ui.Modifier
import androidx.compose.foundation.clickable

@Composable
fun HomeScreen(count: Int) {
    Text(
        text = "Clicks: $count",
        modifier = Modifier.clickable(onClick = {})
    )
}
```

В onClick мы пока указали пустую лямбду - {}. Т.е. ловим клики, но пока ничего не делаем. Давайте думать, что именно нам надо делать.

Мы хотим по клику увеличивать счетчик. А где хранится его значение? В Activity:

```
var count = 0
```

Значит нам надо из Composable функции как-то сообщить Activity, что значение счетчика надо увеличить. Для этого можно использовать колбэк. Activity передает его в Composable функцию, которая вызывает его по клику на тексте. В итоге Activity получит вызов колбэка и сможет увеличить значение счетчика.

Добавляем лямбду-колбэк в параметры функции HomeScreen:

```
@Composable
fun HomeScreen(
    count: Int,
    onCounterClick: () -> Unit
) {
    Text(
        text = "Clicks: $count",
        modifier = Modifier.clickable(onClick = onCounterClick)
    )
}
```

и вешаем эту лямбду в качестве onClick в Text. Теперь по нажатию на текст будет вызываться onCounterClick.

Т.е. Composable функция тут похожа на View в архитектуре MVP или MVVM. Пользователь нажимает что-то на экране, и View сообщает об этом нажатию презентеру или ViewModel, а те уже решают, как на это реагировать. Наша Composable функция тоже сама не меняет значения. Она сообщает Activity, что был клик. А Activity уже будет на это реагировать и увеличивать счетчик.

Подставляем код Activity. В HomeScreen в качестве onCounterClick передаем лямбду, которая увеличивает счетчик:

```
class MainActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        var count = 0
        setContent {
            HomeScreen(
                count = count,
                onCounterClick = {
                    count++
                }
            )
        }
    }
}
```

В итоге событие нажатия на текст приведет к тому, что count увеличится на 1. Но отобразится ли это новое значение счетчика на экране?

Не отобразится. Когда мы вызвали HomeScreen, мы передали туда значение переменной count. То, что мы потом меняем эту переменную в Activity, никак не сказывается на том значении, которое мы уже передали в HomeScreen. В итоге функция просто не получает новое значение count и продолжает отображать изначальное значение 0.

### Контейнер

Попробуем поменять реализацию, чтобы была возможность донести до функции новое значение. Поместим значение счетчика в объект, и передадим этот объект в функцию. По нажатию мы меняем значение счетчика в объекте, и функция точно сможет получить это значение из объекта.

Создадим класс:

```
class Counter {
    var value = 0
}
```

Он играет роль контейнера и содержит в себе значение счетчика.

Перепишем HomeScreen с использованием класса Counter:

```
1 @Composable
2 fun HomeScreen(
3     counter: Counter,
4     onCounterClick: () -> Unit
5 ) {
6     val counterValue = counter.value
7     Text(
8         text = "Clicks: $counterValue",
9         modifier = Modifier.clickable(onClick = onCounterClick)
10    )
11 }
```

Функция принимает на вход объект Counter и отображает значение его поля value. Если мы снаружи меняем это значение в объекте, то функция точно сможет его получить.

Перепишем MainActivity

```
class MainActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        val counter = Counter()
        setContent {
            HomeScreen(
                counter = counter,
                onCounterClick = {
                    counter.value++
                }
            )
        }
    }
}
```

Создам объект Counter и передам его в функцию. А по клику увеличиваем значение в этом объекте.

Теперь казалось бы должно работать. Но охота нет. Это типичная ошибка, с которой вы столкнетесь еще не раз. Мы вроде меняем значение, а Composable функция почему то не подхватывает эти изменения и продолжает отображать старое значение.

Очень важно понимать почему так происходит. Это ключевой момент в Compose. И он звучит достаточно просто. Если мы хотим, чтобы Composable функция отображала новое значение, необходимо вызвать эту Composable функцию еще раз. Только в этом случае мы увидим на экране изменения.

Возникает вопрос: как это сделать? У нас в Activity метод onCreate уже обработал. В нем мы вызвали Composable функцию HomeScreen. Как сделать этот вызов еще раз? Создатели Compose позаботились об этом. Composable функция умеет перезапускать сама себя. Но для этого необходимо использовать специальный контейнер State при передаче данных в Composable функцию.

### State

State похож на объект Counter, который мы пытались сами использовать. Это тоже контейнер, который содержит значение. Но он иначе. Когда мы меняем значение в State, то Composable функция узнает об этом и перезапустится, чтобы считать это новое значение и отобразить его.

Используем State в HomeScreen:

```
import androidx.compose.runtime.State

@Composable
fun HomeScreen(
    counter: State<Int>,
    onCounterClick: () -> Unit
) {
    val counterValue = counter.value
    Text(
        text = "Clicks: $counterValue",
        modifier = Modifier.clickable(onClick = onCounterClick)
    )
}
```

Получаем на вход State объект. Вытаскиваем из него значение и используем в Text. В этот момент и начинает работать магия Compose.

Когда мы в Composable функции читаем значение из State:

```
val counterValue = counter.value
```

функция понимает это и под капотом подписывается на факт изменения значения в этом State, чтобы выполнить перезапуск себя при изменении значения. В результате, когда мы меняем значение счетчика в counter: State<Int>, то Composable функция HomeScreen перезапустится. Это приведет к тому, что она считает из State новое значение в переменную counterValue и перезапустит функцию Text с этим новым значением. В результате чего мы увидим новое значение на экране. И т.д. Но, конечно, функция HomeScreen снова подпишется на изменения в counter.

Это важно понимать. Без перезапуска Composable функции никаких изменений на экране мы не увидим. Вся эта магия с перезапуском обеспечивается Compose компилятором, который добавляет в наши Composable функции специальный код, отвечающий за подписку на State.

На всякий случай проговорю явно, что функция подписывается не на значение State, а на сам факт, что значение изменилось. Т.е. когда значение State меняется, функция получает не новое значение State, а просто уведомление о том, что State изменился. И далее уже функция перезапускает себя, чтобы считать из State это новое значение.

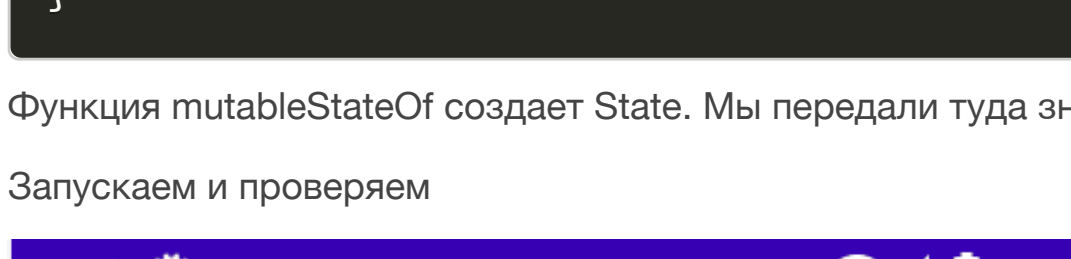
Нам осталось только создать State объект в Activity и передать его в HomeScreen

```
import androidx.compose.runtime.mutableStateOf

class MainActivity() : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        val counter = mutableStateOf(0)
        setContent {
            HomeScreen(
                counter = counter,
                onCounterClick = {
                    counter.value++
                }
            )
        }
    }
}
```

Функция mutableStateOf() создает State. Мы передали туда значение 0. А по клику мы будем увеличивать это значение в State.

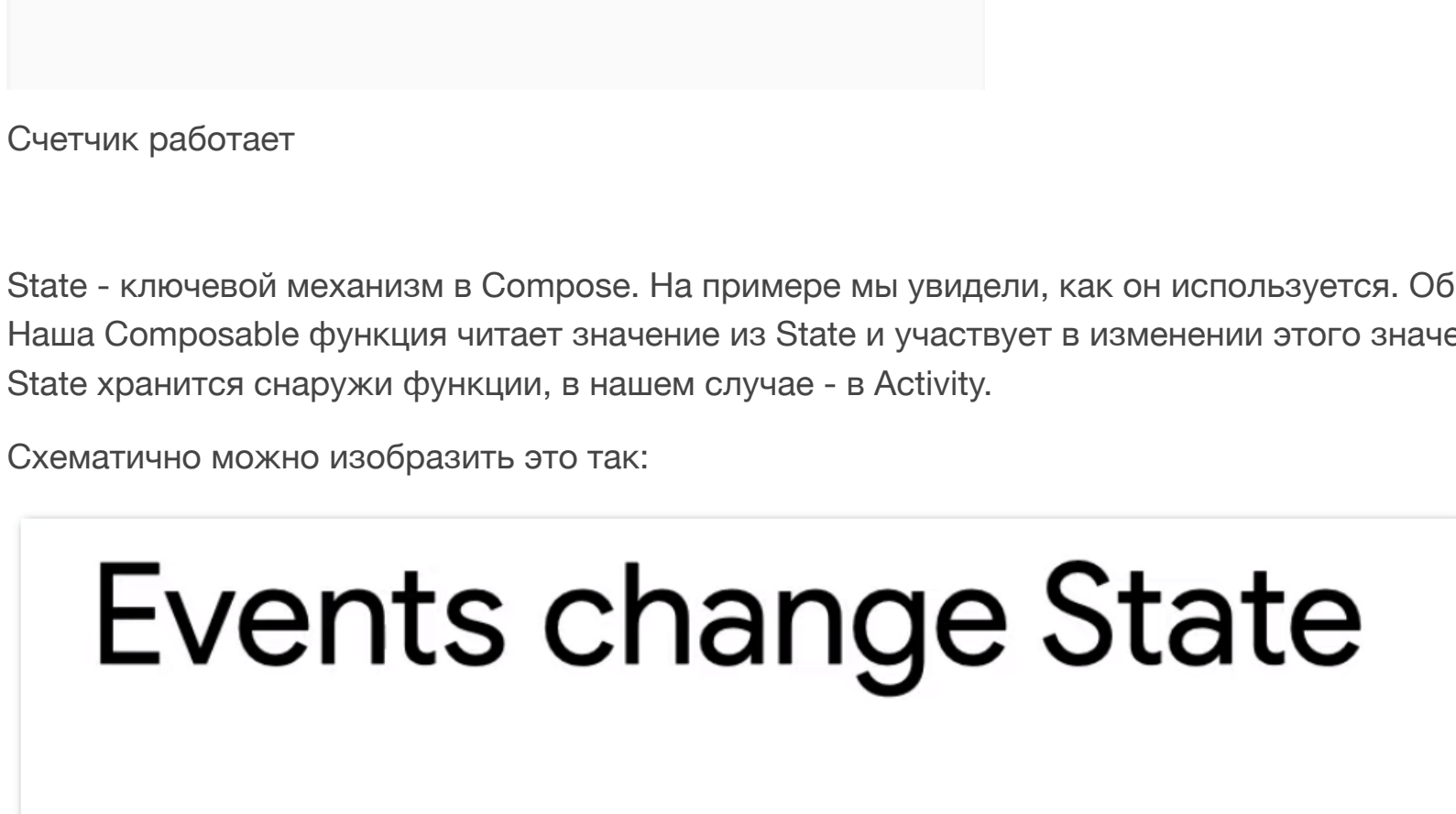
Запускаем и проверяем



Счетчик работает

State - ключевой механизм в Compose. На примере мы увидели, как он используется. Обратите внимание на важный момент. Наша Composable функция читает значение из State и уведомляет в изменении этого значения с помощью колбэка. Но при этом State хранится снаружи функции, в нашем случае - в Activity.

Схематично можно изобразить это так:



Источники

UI - это Composable функция. Она получает на вход State и отображает данные из него. Обратно она шлет действия пользователям, например, клики. Это приводит к изменениям в State, которые снова придут в функцию. И т.д. по кругу.

Основная мысль в том, что State находится снаружи, а не внутри функции. Это стандартный подход в Compose, и называется он - [State hoisting](#). И в этом есть смысл.

Например, мы хотим, чтобы счетчик игнорировал каждое второе нажатие, а максимальное число нажатий было 10. Если мы начнем зашивать эту логику в Composable функцию, то такая функция получится очень не универсальной. Мы уже не сможем ее так просто переиспользовать для отображения других счетчиков, с другой логикой. К тому же мы не сможем покрыть эту логику Unit тестами. Поэтому State и вся логика по его изменению обычно находится снаружи функции.

Но бывают и случаи, когда State можно хранить внутри Composable функции. Такие примеры мы еще рассмотрим в последующих уроках.

### MutableState

Вы скорее всего знаете про пары SharedFlow/MutableSharedFlow, StateFlow/MutableStateFlow и LiveData/MutableLiveData. У State тоже есть свой MutableState. Именно его мы и создавали в Activity методом mutableStateOf(). Это дало нам возможность менять там его значение. А вот в Composable функцию мы этот MutableState передавали, как State, чтобы функция могла только читать значение, но не менять его напрямую.

Присоединяйтесь к нам в **Telegram**:

- в канале [StartAndroid](#) публикуются ссылки на новые статьи с сайта [startandroid.ru](#) и интересные материалы с хабра, [medium.com](#) и т.п.

- в чатах решаем возникающие вопросы и проблемы по различным темам: [Android](#), [Compose](#), [Kotlin](#), [RxJava](#), [Dagger](#), [Тестирование](#), [Performance](#)

- ну и если просто хочется поговорить с коллегами по разработке, то есть чат [Флудильня](#)

### Комментарии

**# State — Alex** 11.02.2024 10:14  
При добавлении "val counter = mutableStateOf(0)" студия ругается на UnrememberedMutableState  
помогла лиambda remember (...)  
Поча не стоит заморачиваться?

Ответить

**# # State — Alex** 11.02.2024 12:02  
на 10 уроке понял, что state объявляем до  
setContent  
val counter = mutableStateOf(0)  
setContent (...)

Ответить

**# Preview — Николай** 11.02.2024 17:21  
На этом уроке не смог создать preview функцию для Home  
Можно подсказку как это сделать?

Ответить

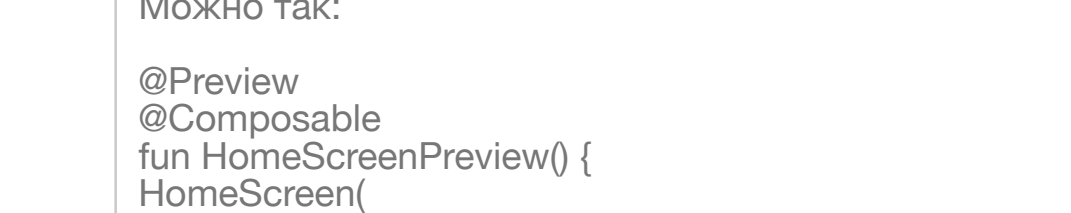
**# RE: Preview — Dmitry Vinogradov** 21.02.2024 18:16  
Можно так:  
@Preview  
@Composable  
fun HomeScreenPreview() {  
 HomeScreen(  
 counter = remember { mutableIntStateOf(1) },  
 onCounterClick = {}  
 )  
}

Ответить

👆 Обновить список комментариев  
📡 RSS лента комментариев этой записи

Добавить комментарий

Тема



Отправить

Comments

